

HW01：Codex 最初の一步（Mac）

模擬教科書の第10章を使い、要約、データを更新したレポート、スライド、四択クイズを作ります。

この手順書に沿って作業する動画：<https://youtu.be/WBNtSK9K48w>

締切と提出物

締切：10月13日（火） 23:59

Google Classroom（クラスコード `jinlu455`）の課題「HW01」に、次の三つを提出します。

- レポート（Word、`.docx`）
- スライド（PDF、`.pdf`）
- クイズ（HTML、`.html`）

ファイル名と提出方法は「10. Google Classroomへ提出する」にあります。

ノート：Claudeを使う場合

Claudeでも、ほぼ同じ手順で進められます。ChatGPTデスクトップアプリの代わりにClaudeデスクトップアプリをインストールし、同じ `HW01` フォルダを開きます。第5節からは、Visual Studio CodeのClaude拡張機能（`anthropic.claude-code`）に移ります。指示文はそのまま使えます。第2節の音声機能だけは飛ばします。

目次

HW01 : Codex 最初の一步 (Mac)

締切と提出物

1. フォルダを準備し、Codexを開く
2. 小さなファイルを作り、プレビューする
3. 第10章をまとめる
4. レポート作成に使うソフトを準備する
5. Visual Studio Codeで同じフォルダを開く
6. 第10章のデータを更新する
7. 第10章のレポートを作る
8. 第10章のスライドを作る
9. 第10章のクイズゲームを作る
10. Google Classroomへ提出する

1. フォルダを準備し、Codexを開く

1. Finderで書類（Documents）フォルダを開きます。その中に **金融論** フォルダを作り、さらに **HW01** フォルダを作ります。
2. 配布された [moc_text.pdf](#) を **HW01** フォルダに入れます。
3. [ChatGPTデスクトップアプリ](#)をMacにインストールします。
4. アプリを起動し、ChatGPTアカウントでサインインします。
5. アプリでCodexを選び、**書類（Documents）/金融論/HW01** を開きます。作業場所はPC上のフォルダを使うLocalです。
6. フォルダへのアクセスを求められたら、対象が **HW01** であることを確認して許可します。

第4節までは、ChatGPTデスクトップアプリのCodexを使います。

指示を複数行で書く

入力途中に送信されるのを防ぐため、アプリの設定で **Enter** / **Return** を改行、**Command + Enter** を送信にします。

2. 小さなファイルを作り、プレビューする

Codexは、指示をもとにファイルを読み書きし、必要な処理を実行します。最初に、入力欄の設定を確認します。

Model・Effort・Speed

- Modelは、作業を行うAIの種類です。Astraは複数の工程を含む難しい作業、Solは日常的な作業や分析、Lunaは手順が明確な繰り返し作業に向きます。
- Effortは、どの程度深く考えるかの設定です。まず画面の初期設定から始めます。高くすると、時間や使用量が増えることがあります。
- Speedは処理速度の設定です。この宿題では **Standard** を使います。 **Fast** は対応するモデルを速く動かしますが、使用量が増えます。

Permissions（権限）

Permissionsは、Codexが作業できる範囲と、追加の操作を誰が承認するかの設定です。

設定	作業できる範囲	追加の操作の承認
<code>Ask for approval</code> （承認を求める：自分で判断）	作業フォルダ内のファイル編集や通常の処理を進めます	インターネット利用や作業範囲を超える操作は、人間に確認します
<code>Approve for me</code> （代わりに承認：AIが判断）	<code>Ask for approval</code> （承認を求める：自分で判断）と同じ作業範囲から始めます	追加の操作を別のAIが審査します。審査を誤る場合もあります
<code>Full access</code> （フルアクセス）	PC上の広い範囲のファイルやインターネットへアクセスします	個々の操作について人間の承認を求めずに進めます

この講義では基本的に `Approve for me`（代わりに承認：AIが判断）を使います。承認画面が表れた場合は、操作の内容と対象を確認します。分からない場合は、その操作が必要な理由をCodexに聞けます。

MarkdownとPDFをプレビューする

Codexに次のように指示します。

仙台市内の美味しい醤油ラーメンのお店を調べて、`preview_test.md`に表でまとめて。

作成された `preview_test.md` を開きます。リンクがでと思うので、それをクリックするとPreview（プレビュー）が開きます。ファイルを完成時に近い見たい目に表示する機能です。見出しと表が表示されるか確認します。

音声機能を使ってみる

醤油ラーメンの表をプレビューしたら、ChatGPTアプリでこのタスクを開きます。入力欄の右側にある縦線が並んだ黒いボタンにカーソルを合わせ、「Start voice chat」と表示されたらクリックします。初回はマイクの使用を許可します。まず「こんにちは」と話しかけ、挨拶が返ってくることを確認します。その後、次のように話しかけます。

`preview_test.md`に仙台市内の美味しい豚骨ラーメンのお店も追加して。

音声認識で「豚骨ラーメン」やファイル名が正しく文字になっているか確認します。Codexが編集したら、`preview_test.md` のプレビューを開き直し、醤油と豚骨の店が表示されるか確かめます。

また、以下のような指示も出してください。教科書のページを確認できます。

教科書のpdfファイルも開いて。

3. 第10章をまとめる

Codexに次のように指示します。

`moc_text.pdf`の第10章「貨幣・信用創造とマネースtock統計」を読み、内容を簡単にまとめた`chapter10_summary`

`chapter10_summary.md` を開き、教科書の内容と要約を比べます。また、テキストエディットで開くと、Markdownの原稿を確認できます。

Markdownは、文章の構造を簡単な記号で表す書き方です。`#` は見出し、`-` は箇条書き、空行は段落の区切りになります。原稿とプレビューを比べると、記号が表示にどう反映されるか分かります。

4. レポート作成に使うソフトを準備する

これから使うソフトの役割は、次のとおりです。

ソフト	この宿題での役割
Homebrew	ソフトのインストールと管理に使います
Python	公開データを取得・整理し、グラフを作ります
Node.js	JavaScriptで作られた道具をPC上で動かします
Marp CLI	MarkdownのスライドをPDFに変換します。Node.js上で動きます
Pandoc	MarkdownのレポートをWordやHTMLに変換します
Visual Studio Code	原稿を開き、編集やプレビュー、Codexへの依頼を行います

ソフト	この宿題での役割
Google Chrome	MarpでスライドをPDFに変換するときに使います。 Safariだとダメらしい。

Homebrewを入れる

Homebrewは、Macのソフトをまとめて管理する仕組みです。最初の導入はターミナルから行います。

1. アプリケーションの「ユーティリティ」から「ターミナル」を開きます。
2. [Homebrew公式サイト](#)のインストール用コマンドをコピーし、ターミナルに貼り付けて実行します。
3. `Password:` と表示されたら、Macにログインするときのパスワードを入力し、Returnを押します。入力した文字は表示されません。
4. `Press RETURN to continue` と表示されたらReturnを押します。Command Line Toolsの導入画面が出たら、案内に沿ってインストールします。
5. 完了時に `Next steps` が表示された場合は、そこにある設定を実行します。

Codexで、Homebrewが使えるか確認します。

Homebrewがインストールされ、使える状態か確認して。設定が残っていれば、その手順を教えて。

必要なソフトを入れる

Codexに次のように指示します。

Homebrewを使ってGit、Python、Node.jsのLTS版（長期サポート版）、Pandoc、Visual Studio Codeをインストール。
Google Chromeが入っていないければ、Google Chromeもインストールして。
手動操作が必要な場合は、その手順を教えて。
それぞれが使えることを確認して。

インストール後のソフトが見つからない場合は、アプリを開き直すと反映されることがあります。手動のインストール画面が出た場合は、画面の案内に沿って進めます。

MarpとPythonの環境を準備する

npmを使ってMarp CLIをインストールし、使えることを確認して。

Pythonで使うライブラリは、宿題フォルダの外にある `finance-env` にまとめます。後の講義で解説します。

書類 (Documents) / 金融論 / `finance-env` にPythonの仮想環境を作って。

その仮想環境にpandas、matplotlib、requestsをインストールし、すべて読み込めることを確認して。

以後、この宿題のPython作業では書類 (Documents) / 金融論 / `finance-env/bin/python` を使って。

`finance-env` は、この宿題で使うPythonの実行環境とライブラリをまとめたフォルダです。

5. Visual Studio Codeで同じフォルダを開く

Visual Studio Codeを開き、Codexで使っている `書類 (Documents) / 金融論 / HW01` を選びます。

`moc_text.pdf` と `chapter10_summary.md` が一覧に並びます。

拡張機能は、Visual Studio CodeにAIとの対話やスライド作成などの機能を追加するためのものです。

左側のパネルにある「拡張機能」アイコンをクリックすると、拡張機能画面が開きます。この画面から、次をインストールします。

- `openai.chatgpt` (Codex)
- `marp-team.marp-vscode` (Marp)

Codexのアイコンを開き、ChatGPTアカウントでサインインします。ここでのサインインはCodexを使うためのものです。[OpenAI DocsのIDE拡張機能](#)

ここからは、Visual Studio CodeのCodexを使います。

原稿と保存の印を見る

`chapter10_summary.md` を開き、`Command + Shift + V` でプレビューを表示します。マウスで操作する場合は、画面上部の `chapter10_summary.md` のタブを右クリックし、「プレビューを開く (Open Preview)」を選びます。元の文章と、見出しや箇条書きの表示を比べます。

手作業で編集した内容が未保存の場合、タブには●が表示されます。`Command + S` で保存すると、変更がファイルへ反映されます。保存してからCodexに依頼すると、編集した内容を読み取れます。

PDFはMacのプレビューで開けます。VS Code内で表示する場合は、PDF表示用の拡張機能 `vscode-pdf` を使えます。

ファイルや行をCodexに見せる

見てほしい行を選び、右クリックの `Add to Codex Thread` から会話に追加します。ファイル全体は、タブの `Add File to Codex Thread` から追加できます。メニューが見つからない場合は、ファイル名と対象の見出しを指示に書くと、場所を伝えられます。

`Add File to Codex Thread` には、キーバインド（ショートカットキー）を割り当てておくとう便利です。設定方法はCodexに聞けます。

6. 第10章のデータを更新する

図10-3「マネタリーベースとマネーストックの伸び」を、日本銀行の最新の公式データで作り直します。

`moc_text.pdf`の図10-3「マネタリーベースとマネーストックの伸び」を、日本銀行の最新の公式データで更新したい。データ、取得・作図コード、更新したグラフ、データの出所と取得日を記録したMarkdownを`data_update`フォルダにま

`data_update` のグラフと出所を開きます。対象期間と単位を見ると、どのデータで更新されたか分かります。

7. 第10章のレポートを作る

Markdownの原稿からWordとHTMLを作ります。文章を直す場所は `chapter10_report.md` です。原稿を直した後は、WordとHTMLも作り直すと内容が揃います。

`moc_text.pdf`、`chapter10_summary.md`、`data_update`フォルダの更新した図表を使って、第10章の短いレポートを
原稿は`chapter10_report.md`、AIに依頼した作業の記録は`AI_LOG.md`にまとめて。
Pandocを使って`output`フォルダに`chapter10_report.docx`と`chapter10_report.html`も作って。

次のファイルを開いて確認します。

- `chapter10_report.md` : レポートの原稿です。
- `AI_LOG.md` : 依頼した作業の記録です。
- `output/chapter10_report.docx` : 提出するWordレポートです。
- `output/chapter10_report.html` : ブラウザで読めるレポートです。

コラム：LibreOfficeを使う方法もあります

オープンソースのオフィススイートであるLibreOfficeを導入すると、WordファイルをPDFに変換し、Codexにレイアウトの確認を依頼することもできます。導入や使い方はCodexに聞けます。MS Wordとは表示が異なる場合があるため、提出前にはMS Wordでも確認します。

8. 第10章のスライドを作る

スライドの原稿は、レポートとは別の `chapter10_slides.md` にします。MarpはMarkdownからスライドを作る仕組みです。本文中の `---` がページの区切りになります。

`moc_text.pdf`、`chapter10_summary.md`、`data_update`フォルダの更新した図表を使って、第10章のスライドを作った。Marp形式の`chapter10_slides.md`と`chapter10_slides.pdf`を作り、表示も確認して。

VS Codeで `chapter10_slides.md` の原稿とプレビューを見ます。提出用の `chapter10_slides.pdf` も開き、図や文字が収まっていることを確認します。

9. 第10章のクイズゲームを作る

教科書の内容を復習する四択クイズを10問作り、一つのHTMLファイルに保存します。クイズも提出物です。

`moc_text.pdf`と`chapter10_summary.md`を使って、第10章を復習する四択クイズゲームを作った。 `quiz.html`という

`HW01` のファイル一覧で `quiz.html` が保存されたことを確認します。

10. Google Classroomへ提出する

締切：10月13日（火） 23:59

提出する三つのファイルをコピーし、次の名前を付けます。

提出物	元のファイル	提出するファイル名
レポート	<code>output/chapter10_report.docx</code>	<code>学籍番号_好きな曲.docx</code>
スライド	<code>chapter10_slides.pdf</code>	<code>学籍番号_好きな漫画.pdf</code>

提出物	元のファイル	提出するファイル名
クイズ	quiz.html	学籍番号_好きな映画.html

例えば、次のようになります。

12345678_レミオロメン3月9日.docx
 12345678_僕らはみんな河合荘.pdf
 12345678_ヴァイオレットエヴァーガーデン.html

1. Google Classroomにクラスコード `jinlu455` で参加します。
2. 課題「HW01」を開きます。
3. 「+ 追加または作成」から「ファイル」を選び、名前を付けたWord・PDF・HTMLを添付します。
4. 三つが添付されたことを確認し、「提出」を押します。

原稿やデータ、コード、`AI_LOG.md` は、引き続き `HW01` フォルダに保存しておきます。